Solution: Valid Palindrome

Problem Statement

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` if it is a palindrome, or `false` otherwise.

Example 1:

```
Input: sentence = "A man, a plan, a canal, Panama!"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: sentence = "Was it a car or a cat I saw?"
Output: true
Explanation: Explanation: "wasitacaroracatisaw" is a palindrome.
```

Example 3:

```
Input: sentence = "race a car"
Output: false
Explanation: Explanation: "raceacar" is not a palindrome.
```

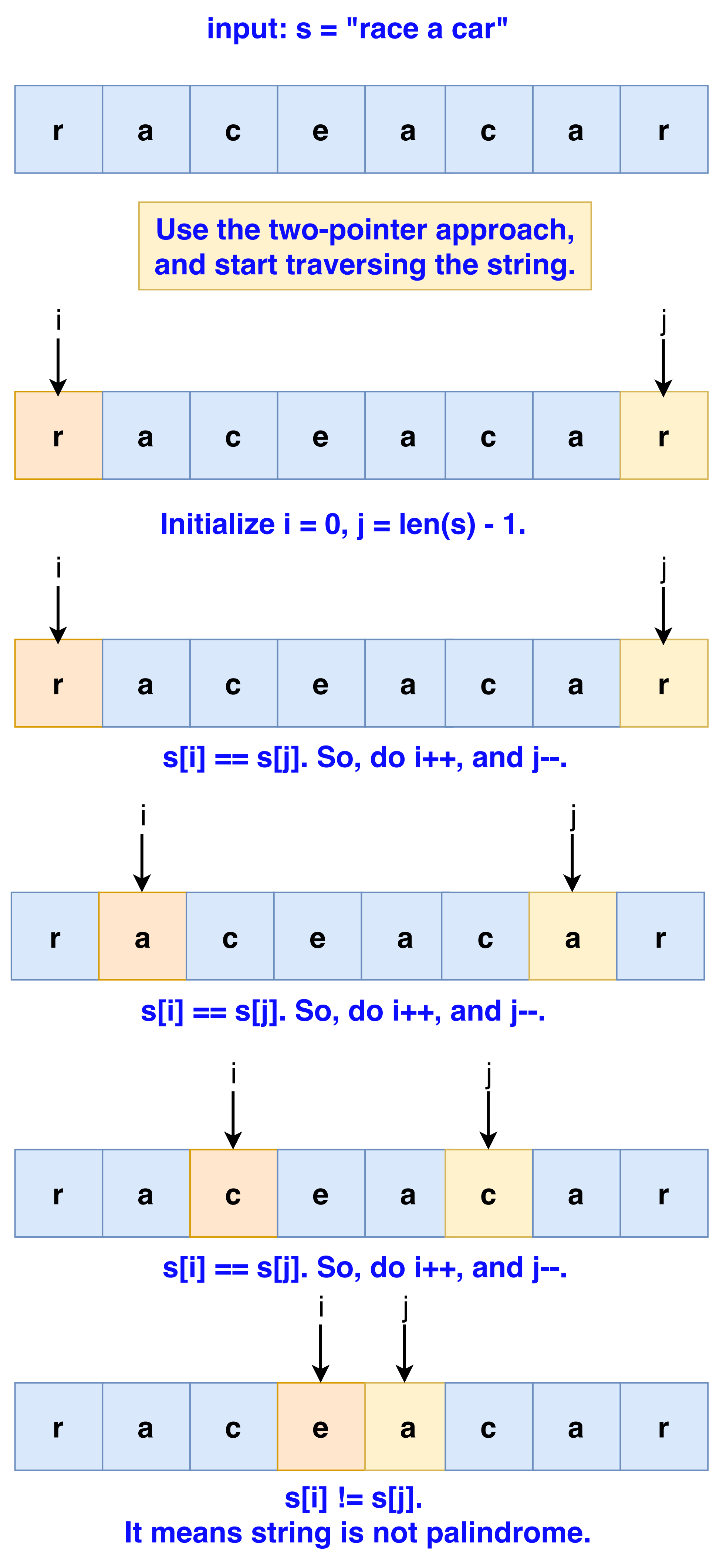
Solution

This problem aims to check if a given string is a palindrome. A palindrome is a word, phrase, number, or other sequences of characters that read the same forward and backward, ignoring spaces, punctuation, and capitalization. Our algorithm can leverage the two-pointer technique where one pointer starts at the beginning of the string, and the other one starts at the end. The two pointers move towards each other, checking if the characters they point to are the same.

Walkthrough of the algorithm

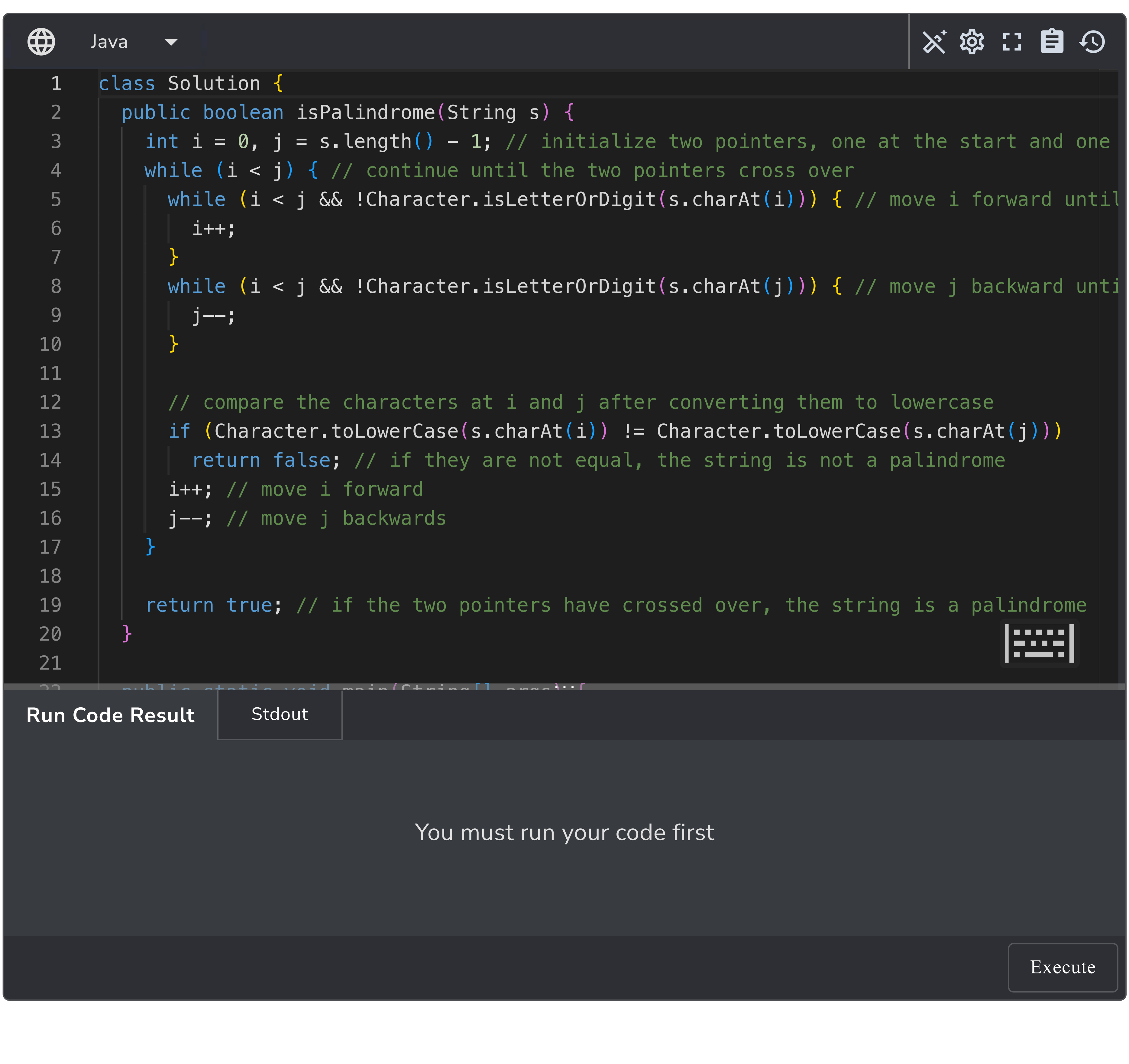
Let's take a detailed walkthrough of the code:

1. The `isPalindrome` function starts by setting two pointers: 'i' at the start of the string and 'j' at the end.
2. Then it enters a while loop which continues until the two pointers cross each other. Inside this loop:
    - The first inner while loop moves 'i' forward, skipping any characters that are not letters or digits, until it points to a valid character or it crosses 'j'.
    - The second inner while loop moves 'j' backward, skipping any characters that are not letters or digits, until it points to a valid character or it crosses 'i'.
3. After finding valid characters at 'i' and 'j', it converts these characters to lowercase and checks if they're the same.
    - If they're not the same, the function immediately returns `false` since the string can't be a palindrome if these characters are different.
    - If they are the same, 'i' is incremented and 'j' is decremented, and the loop continues to the next pair of characters.
4. If the while loop completes without finding any unequal pairs of characters, the function returns `true`, indicating that the string is a palindrome.


input: s = "race a car"
r a c e a c a r
Use the two-pointer approach, and start traversing the string.
i
j
r a c e a c a r
Initialize i = 0, j = len(s) - 1.
i
j
r a c e a c a r
s[i] == s[j]. So, do i++, and j--.
i
j
r a c e a c a r
s[i] == s[j]. So, do i++, and j--.
i
j
r a c e a c a r
s[i] == s[j]. So, do i++, and j--.
i
j
r a c e a c a r
s[i] != s[j].
It means string is not palindrome.

Code

Here is the code for this algorithm:



```java
class Solution {
  public boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1; // initialize two pointers, one at the start and one
    while (i < j) { // continue until the two pointers cross over
      while (i < j && !Character.isLetterOrDigit(s.charAt(i))) { // move i forward until
        i++;
      }
      while (i < j && !Character.isLetterOrDigit(s.charAt(j))) { // move j backward unti
        j--;
      }

      // compare the characters at i and j after converting them to lowercase
      if (Character.toLowerCase(s.charAt(i)) != Character.toLowerCase(s.charAt(j)))
        return false; // if they are not equal, the string is not a palindrome
      i++; // move i forward
      j--; // move j backwards
    }

    return true; // if the two pointers have crossed over, the string is a palindrome
  }

```

You must run your code first

Time Complexity

The time complexity of this algorithm is $O(n)$, where n is the number of characters in the sentence, because it iterates over each character once.

Space Complexity

The space complexity is $O(1)$ because our algorithm uses constant space to store a few variables, and the size of these variables does not grow with the input size.

← Previous
Valid Palindrome (Easy)
Next →
Valid Anagram (Easy)
☐ Mark as Completed